

SemantOS: Toward Auditable, Knowledge-Grounded Kernel Tuning with Bundle-Level Risk Gating

Anonymous ACL submission

Abstract

Language models are increasingly proposed as kernel and database tuners, yet their value over an equally informed optimizer, and the safety of applying their proposals, are rarely measured with independent outcomes. We present SemantOS, a control loop that separates evidence (a typed interaction graph and retrieved traces), proposals (a retrieval-augmented reasoner emitting multi-knob bundles), and enforcement (a runtime that validates, vetoes or stages a complete bundle). We make three contributions. First, we specify a bundle-level control contract and implement it as a dry-run REST prototype together with a separate bounded per-thread actuator. Second, we give an expected-cost identity for gated acceptance and a class-conditional rank gate whose conditioning and exchangeability assumptions are explicit. Third, we define a role-separated evaluation protocol with raw-log verification and apply it to Linux timer-slack/affinity interventions (8,000 grid events, 80 staged executions). Equally informed non-LLM selectors choose the default configuration, and matched-budget BO and RL baselines converge on it, so held-out replay shows no improvement; a frozen gate accepts 5 of 25 and 4 of 24 unsafe runs under a period shift, exceeding its nominal level; and all 80 staged executions restore both original settings. These results show that the protocol exposes unfavorable outcomes rather than hiding them. Ten LLMs, from 7B open-weight models to hosted frontier models, given the same frozen evidence do not improve on the default (one is worse on replay), and their explanations often misstate that evidence despite valid citations.

1 Introduction

Kernel tuning requires choosing configurations whose effects depend on workload, hardware and the current operating state, and a setting that improves one metric can worsen another. Learned

tuners have long been applied to databases and data-processing systems (Van Aken et al., 2017; Zhang et al., 2019; Herodotou et al., 2020), and language models now read manuals, propose complete configurations, or search kernel build options (Trummer, 2022; Lao et al., 2024; Giannakouris and Trummer, 2025; Chen et al., 2024; Lin et al., 2025). Two questions are nevertheless hard to answer from existing evaluations. Does a language model add value beyond the information already present in its knowledge base (KB) when a conventional optimizer receives the same inputs? And when a proposal is applied, is the risk of harm measured by an outcome that does not depend on the enforcement mechanism itself?

SemantOS is designed to make both questions testable. It separates three responsibilities: evidence acquisition, candidate generation and enforcement. A typed graph and retrieved traces expose the available evidence; a reasoner proposes a multi-knob bundle; and a runtime checks the complete bundle and decides whether to veto it or enter a staged rollout. Because the evidence is explicit, the same evidence can be handed to a non-LLM optimizer; because enforcement is independent, its errors can be counted against labels defined from raw observations.

Contributions. (i) A bundle-level control contract, with a dry-run REST implementation and a separate actuator that applies, verifies and restores per-thread kernel settings (§4). (ii) An expected-cost identity for gated acceptance and a restricted rank gate, stated with the conditioning and assumptions under which they hold (§4.2). (iii) A role-separated evaluation protocol that freezes selections and thresholds before test data are opened and reports every denominator, applied to real Linux interventions (§5 and §5.4). The measured results are deliberately reported even where unfavorable: informed non-LLM selectors and matched-budget

BO/RL baselines yield no gain over the default, ten LLMs, including hosted frontier models, do no better, and the frozen gate misses more unsafe runs than its nominal level under a period shift.

Scope. The REST runtime never writes kernel settings, and the actuator changes only an owned child thread; the claims this scope does and does not support are collected in the Limitations section.

2 Background and Related Work

Configuration tuning for systems. OtterTune (Van Aken et al., 2017) reuses past workload observations with Gaussian-process models to tune database knobs, and CDBTune (Zhang et al., 2019) learns a deep reinforcement-learning policy from trial rewards; a survey (Herodotou et al., 2020) covers rule-based, cost-model, experiment-driven, learned and adaptive tuners for data-processing systems. Bayesian optimization (Snoek et al., 2012) and services such as Vizier (Golovin et al., 2017) are general black-box baselines. For the Linux kernel, STUN (Lee et al., 2022) tunes scheduler parameters with Q-learning for static workloads. None of these methods must assume independent parameters: a surrogate can model interactions, and a graph-aware optimizer can receive exactly the evidence given to an LLM. A comparison that grants offline pairwise-search results only to the language-model system therefore cannot isolate language-model reasoning; our protocol includes both cold-start and shared-KB optimizers and accounts separately for offline and online costs.

Language models for configuration. DB-BERT (Trummer, 2022) mines tuning hints from manuals and refines them with trial runs; GPT-Tuner (Lao et al., 2024) uses GPT-extracted knowledge to prune knobs and ranges before coarse-to-fine Bayesian optimization; and λ -Tune (Giannakouris and Trummer, 2025) generates complete configuration scripts and bounds the cost of evaluating candidates. For operating systems, AutoOS (Chen et al., 2024) uses an LLM-driven observe-prune-propose-act-correct loop over Linux kernel build options, and TuneAgent (Lin et al., 2025) trains an LLM agent with rule-based reinforcement learning over Kconfig option groups. These systems target build-time or database configuration and evaluate end-to-end performance. SemantOS instead targets runtime

controls and focuses on the enforcement boundary: complete bundles are validated and gated by a component independent of the proposer, and the evaluation separates outcome labels from enforcement decisions. We do not claim to outperform these systems; no head-to-head comparison is reported.

Grounding and uncertainty. Retrieval-augmented generation (Lewis et al., 2020) conditions outputs on retrieved documents, and self-consistency (Wang et al., 2023) aggregates multiple sampled outputs. SemantOS uses both, but treats agreement among samples only as a heuristic score to be calibrated, not as a probability.

Risk control. Selective classification (Geifman and El-Yaniv, 2017) abstains to meet a target risk. Conformal Risk Control (Angelopoulos et al., 2024) controls expected monotone losses, and its non-exchangeable extension (Farinhas et al., 2024), like weighted conformal prediction (Barber et al., 2023) and adaptive conformal inference (Gibbs and Candès, 2021), addresses drifting data with explicit weights or online updates. None implies that an arbitrary sliding window restores a fixed risk level after a change detected by, e.g., ADWIN (Bifet and Gavalda, 2007). Our rank gate is a restricted class-conditional result, not a replacement for these frameworks or a new drift guarantee.

Explanation faithfulness. Provenance lists what context was supplied, not necessarily what influenced a recommendation; faithfulness must be evaluated separately from plausibility (Jacovi and Goldberg, 2020). ContextCite (Cohen-Wang et al., 2024) attributes generated output to context by ablation. We adopt matched context-deletion interventions to test whether cited edges and traces influence the proposed bundle; operator preference alone cannot establish faithfulness.

Kernel interfaces. Linux documents runtime controls such as `vm.swappiness`, `vm.dirty_ratio` and `vm.dirty_background_ratio` (Linux kernel contributors, 2024), and per-thread controls such as timer slack and CPU affinity (Linux man-pages project, 2026a,b). Availability and semantics must be checked on the running kernel; the example policy uses a small allowlist of documented controls.

3 Motivation and Testable Questions

SemantOS is motivated by three hypotheses. First, a configuration’s utility may depend on workload and hardware. Second, joint knob effects may be non-additive. Third, performance, failure probability and intervention cost may conflict. Each hypothesis requires measured interventions with explicit baselines.

For a loss L , define an interaction contrast from four independently measured configurations:

$$I_{ab} = L(a, b) - L(a, b_0) - L(a_0, b) + L(a_0, b_0).$$

For lower-is-better loss, negative I_{ab} indicates a beneficial non-additive interaction relative to the chosen baseline. A bootstrap interval must resample independent runs, not individual requests from a single run. The sign can change with workload, value range or kernel version, so an edge needs its scope, configuration values, measurement IDs and uncertainty interval. Such an edge describes an empirical interaction, not an unconditional causal coupling.

A DEPENDS-ON edge requires a validated admissibility rule or measured range conditioning; it cannot be inferred solely from the sign of I_{ab} . The released seed graph is empty until edges pass this test. The principal question is whether a reasoner can use independently acquired evidence more effectively than an optimizer with the same inputs, after charging the cost of acquiring that evidence.

4 SemantOS Design

4.1 Control Contract

The architecture contains telemetry, a graph and trace store, a reasoner, a runtime and an operator console. The REST prototype connects these components, but its runtime is a *dry run*: it stages an in-memory bundle and never writes a kernel setting, routes traffic or demonstrates a kernel rollback.

Evidence and score. The reasoner retrieves typed neighbors and lexically matched trace text (Lewis et al., 2020), then requests $k = 3$ model samples (Wang et al., 2023). For a knob–value pair proposed in v samples, its heuristic score is $u = 0.7(1 - v/k) + 0.3(1 - w)$, clipped to $[0.02, 1]$, where w is the strongest retrieved synergy weight (zero when absent). This is neither predictive entropy nor a calibrated probability. Backend failure must be recorded; repeated copies of a failure do

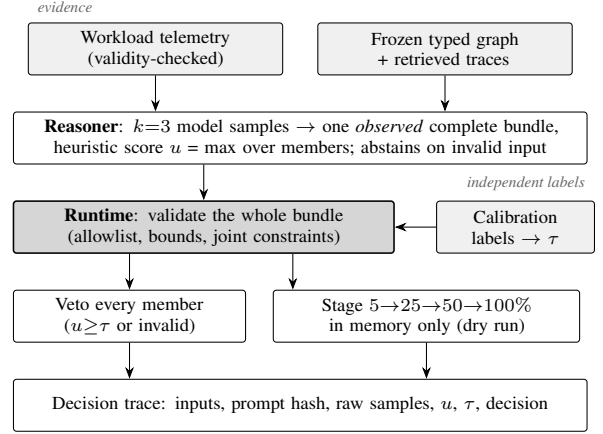


Figure 1: SemantOS decision path as implemented. Evidence, proposal and enforcement are separate components, so the same evidence can be given to a non-LLM optimizer and gate errors can be counted against independently defined labels. The REST runtime never writes kernel settings; a separate per-thread actuator (evaluated later) applies, verifies and restores timer slack and affinity in owned child processes.

not constitute independent model votes. The implementation abstains when no configured model produces recommendations. Supplied evidence is recorded as provenance, not certified as a faithful explanation.

The aggregator chooses a modal complete bundle actually present in a model response, rather than splicing independently winning knob values into an unseen combination. Backend failures retain their type and failed samples remain in the score denominator. Responses expose the complete supplied context, prompt hash and raw samples; exceeding the context budget causes abstention instead of truncating evidence. A model name in this record does not by itself establish immutable checkpoint provenance.

Bundle contract. A request contains a nonempty bundle identifier and a list of recommendations with id, knob, proposed, uncertainty, and optional rationale and provenance. Each request submits one bundle. Duplicate knobs, nonfinite scores, unsupported controls, and example-policy bound violations reject the entire request. Dirty-ratio changes must supply both values and satisfy the example policy that the background ratio is less than the foreground ratio. This is a declared policy, not proof that a configuration improves any workload. If any score satisfies $u \geq \tau$, all members are vetoed; otherwise all are staged together. A concurrent bundle receives a conflict response. Dry-run

Algorithm 1 Implemented dry-run bundle path

Require: Validated bundle B , fixed score function, calibration threshold τ

- 1: Validate every member and joint constraints before changing state
 - 2: **if** another bundle is active **then**
 - 3: Return conflict
 - 4: **end if**
 - 5: **if** $\max_{a \in B} u(a) \geq \tau$ **then**
 - 6: Record veto for all members; return
 - 7: **end if**
 - 8: Stage the complete bundle in memory at 5%
 - 9: **for** requested stage in $\{25, 50, 100\}\%$ **do**
 - 10: Advance the simulated bundle; record the transition
 - 11: **end for**
 - 12: On cancellation, clear the complete simulated bundle
 - 13: Never infer an unsafe/safe training label from simulated completion
-

results return `applied=[]` and list simulated members separately.

Deployment requirements. A real actuator must snapshot all old values, validate the full target configuration and available host interfaces, serialize updates, verify readback, and restore every old value on failure. Multiple `sysctl` writes are not a kernel transaction: partial exposure must be measured and cannot be described as atomic hardware state. Canary shares require separate hosts or an appropriate isolation mechanism; changing a host-global `sysctl` cannot isolate a fraction of requests on that host. Missing, stale or malformed end-to-end telemetry must stop promotion. These capabilities are requirements, not features demonstrated by the dry-run state machine.

One illustrative decision trace. Consider supplied context with example values 5 and 15 for the two dirty ratios, and retrieved evidence IDs e_1, t_1 . Suppose all three model samples agree on this pair and $w = 0$, giving $u = 0.30$. If an independently obtained threshold is $\tau = 0.25$, the whole bundle is vetoed; if $\tau = 0.35$, it can enter the dry-run canary after joint validation. The audit record includes the values, scores, threshold and decision. These numbers illustrate the contract and are not a measured recommendation or calibrated operating point. The initial artifact threshold is zero until calibration data is explicitly supplied. 334

4.2 Probability Statements for Gated Acceptance

Let $Y = 1$ denote an independently defined unsafe outcome and $A = 1$ acceptance. An outcome label is not defined by whether a rollback happened. For example, a benchmark can predeclare a request deadline, count observed failures or deadline exceedances, and define an action-level breach from these raw observations in a fixed window. Rejected actions have unknown counterfactual outcomes unless evaluated in an isolated experiment; they must not automatically be labeled safe or unsafe.

Theorem 1 (Cost identity). *For costs $c_{FN}, c_{FP}, \lambda \geq 0$, define $C = c_{FN}\mathbf{1}\{Y = 1, A = 1\} + c_{FP}\mathbf{1}\{Y = 0, A = 0\} + \lambda D$, where $D \geq 0$ is measured exposure or SLO debt. Write $\pi = \Pr(Y = 1)$, $q = \Pr(A = 1 \mid Y = 1)$ and $r = \Pr(A = 0 \mid Y = 0)$. Then*

$$\mathbb{E}C = c_{FN}\pi q + c_{FP}(1 - \pi)r + \lambda \mathbb{E}D.$$

For a zero-probability class its associated product is zero.

Proof. Expand each indicator expectation as its joint probability and use the definition of conditional probability. Linearity requires no independence. $\square$

This is an accounting identity, not a claim of novelty or a safety bound. A bound $q \leq \alpha$ controls misses among unsafe actions. A different bound $\Pr(Y = 1 \mid A = 1) \leq \alpha$ instead gives $\Pr(Y = 1, A = 1) \leq \alpha \Pr(A = 1)$, not $\pi\alpha$. Likewise r is a veto rate among safe actions, not the proportion of vetoed actions that are safe. We report all denominators separately.

Proposition 1 (Restricted rank gate). *Fix a score function before calibration. Suppose n unsafe calibration scores and the next unsafe test score are jointly exchangeable. Set $j = \lfloor \alpha(n + 1) \rfloor$ for $0 < \alpha < 1$. If $j = 0$, veto all; otherwise let τ be the j th smallest unsafe calibration score, and accept only when $u < \tau$. Then the unsafe-class acceptance probability, marginal over calibration and test scores, is at most $j/(n + 1) \leq \alpha$.*

Proof. With distinct scores, the test score has a uniform rank among $n + 1$ exchangeable scores. It lies strictly below the j th calibration order statistic only in the first j ranks. Ties with strict acceptance can only reduce this probability; $j = 0$ accepts nothing. $\square$

For bundle-level calibration, u denotes the maximum member score and Y the independently observed bundle outcome. Per-knob labels alone do not establish a bundle-level guarantee.

Selecting a smaller threshold by a cost rule cannot enlarge the acceptance event. Selecting a larger threshold can invalidate the bound; therefore the implementation uses the minimum of the rank threshold and optional cost-selected cap. No unsafe calibration examples implies veto-all. Calibration-window diagnostics are not test-set estimates.

Drift and selective observation. Sliding windows and a drift detector (Bifet and Gavaldà, 2007) do not guarantee exchangeability or a deterministic recovery time. Under an explicitly justified total-variation bound on the *joint calibration-test law*, the probability of the same event changes by at most that bound; a marginal score-distance statement alone does not establish it. We neither estimate that distance nor claim recovery. Online labels selected by acceptance introduce additional selection bias. The runtime holds its threshold at zero on detected change until an explicit refresh; refresh remains an exploratory operation, not proof of safety.

5 Measurements and Evaluation Protocol

5.1 New Local Measurements

We executed three bounded microbenchmarks on the available host: an Intel Core i5-3570 with four logical CPUs, approximately 31.2 GiB RAM, and Linux 6.17.0-23-generic. These runs characterize the measurement pipeline: no kernel knob was modified and no LLM or controller was compared.

Each workload had 10 separate invocations of 100 measured operations, preceded by 10 warm-up operations. Workload-run order was shuffled with seed 4088. SQLite inserts a 4 KiB value and commits each transaction in WAL mode with synchronous=FULL. The file test overwrites a 64 KiB region and calls fsync. The compression test compresses a fixed seeded 64 KiB payload at zlib level 6 and verifies decompression. Latency is wall-clock operation duration from a monotonic nanosecond clock; the compression measurement includes verification. The raw log retains every operation, run ID, deadline and error flag, with a SHA-256 hash and software/environment manifest.

The anomaly denominator is all measured operations, including failures; an operation counts once if either it fails or exceeds the deadline. This label

Table 1: Measured local baseline. Mean of run medians, and mean of run P95 values with 95% Student- t CI half-width across 10 runs. Last column is the percentage of errors or operations exceeding a predeclared 50 ms deadline. No tuning gain is implied.

Workload	Median ms	P95 ms	Events %
file_fsync	1.903	2.035 ± 0.043	0.00
sqlite_commit	1.870	2.187 ± 0.273	0.00
zlib_compress	1.474	2.038 ± 0.356	0.00

is independent of veto and rollback. Zero events in these short runs is not evidence of zero population risk. We do not pool heterogeneous workload latencies or infer kernel tuning effects from these baseline timings.

5.2 Measured Per-Process Kernel Interventions

We additionally executed real Linux timer-slack and CPU-affinity interventions in fresh owned child processes, without modifying host-wide sysctls. The two factors were timer slack (50 μ s or 1 ms, set by PR_SET_TIMERSLACK) and affinity (all four available CPUs or CPU 0). Every child checked kernel readback before recording events. This isolated measurement driver is separate from the dry-run SemantOS actuator.

For each periodic wake-up interval (2 or 5 ms), five training blocks and ten subsequent test blocks evaluated all four configurations. Within each block, the eight interval/configuration cases were randomly ordered using seed 4088. Each fresh process performed ten warm-up and 80 measured wake-ups against absolute monotonic-clock targets. The plan was written before execution; the configuration minimizing mean training-run P95 lateness was frozen before test execution. This is temporal holdout within one host and workload family, not the independent workload/hardware split required for claims of generalization. The complete 120-process experiment retained 9,600 event timestamps, readbacks, frozen selections and hashes. No external competing load was introduced.

The training selector chose the explicit control (50 μ s slack, all CPUs) at both intervals: there is no selected-policy improvement over that control. Its test P95 was 89.5 ± 9.2 and 94.2 ± 6.9 μ s at 2 and 5 ms, respectively (95% Student- t half-width across ten runs). Raising slack to 1 ms with all CPUs increased P95 by 927.3 ± 16.4 and 928.4 ± 10.0 μ s in paired blocks. Affinity-only differences

Table 2: Held-out kernel intervention measurements. P95 is the mean of ten run-level P95 wake-up latenesses. Misses count lateness exceeding the predeclared 200 μ s deadline. CPU affinity is all four CPUs or one CPU.

Period	Slack/CPU	P95 (μ s)	Misses
2	50us/all	89.5	2/800
2	50us/one	109.4	18/800
2	1ms/all	1016.8	800/800
2	1ms/one	968.9	771/800
5	50us/all	94.2	2/800
5	50us/one	133.8	22/800
5	1ms/all	1022.6	663/800
5	1ms/one	1073.7	680/800

were 19.9 ± 36.7 and 39.6 ± 62.6 μ s; these intervals include zero. The slack effect is consistent with timer coalescing; no power measurement, graph discovery, LLM advantage or staged rollback is inferred. A miss is defined by observed wake-up time, independently of any safety gate.

5.3 Required Controlled Comparisons

Matched information and budgets. An LLM’s incremental value is tested against the unchanged kernel, conventional optimizers with and without the same graph and traces, and a deterministic graph policy, with fixed knob values, workloads, hardware, trial and time budgets, safety policy and objective weights. Model inference, graph construction, retrieval, calibration and setup costs are reported separately, and ablations remove graph, retrieval or the LLM while keeping the rest. Section 5.4 executes this design for one small task.

Leakage, outcomes and aggregation. Workload, hardware and session groups are fixed before graph induction and assigned to disjoint train, retrieval, calibration and test roles; hashes of graphs, sources, prompts and calibrators are frozen before test results are opened, and a validator rejects overlap or unknown lineage. For each bundle we count proposed, rejected, vetoed, accepted, staged, completed and rolled-back outcomes, report gate TP/FP/FN/TN with both $\#$ unsafe-accepted/ $\#$ unsafe and $\#$ accepted, and leave unknown labels unknown; rollback precision needs independently labeled executed actions. A lower threshold accepts a subset of fixed candidates, but the unsafe fraction among survivors need not be monotone. Absolute per-workload metrics precede ratios against a named baseline; Pareto claim needs several measured operating points per method. Drift studies must keep the threshold tra-

jectory and held-out outcomes at each decision, and faithfulness is tested by deleting cited and matched uncited context under fixed seeds.

5.4 Prospective Role-Separated Follow-up

To exercise the protocol’s role separation, frozen selection and independent labels, we collected a separate prospective dataset using the same four timer-slack/affinity configurations. The plan was saved before acquisition. Training, retrieval, calibration and test used periods of 2, 3, 4 and $\{5, 8\}$ ms, respectively. Each period had ten randomized four-configuration blocks, with five warm-ups and 40 recorded events per fresh child. This produces 200 measurement processes and 8,000 events. Roles are period-disjoint, not workload-family- or hardware-independent: all runs use the same periodic workload on one shared host. The acquisition manifest validates declared groups, run hashes and graph/calibrator lineage; the analyzer additionally checks freeze timestamps, record completeness and kernel readbacks.

Shared-information non-LLM comparison.

The empirical selector minimizes mean training-run P95 over the four configurations. A saturated two-factor representation encodes the same four means as a baseline, two main effects and their interaction; minimizing it adds no information to table lookup. Both selectors have the same 40 training runs and zero online search trials. An unchanged-control selector and a seeded uniform-random selector provide references. Choices are frozen before calibration and test. We evaluate these fixed selectors by paired *offline replay* on the same measured test grid, not by separate closed-loop executions. A block is the uncertainty unit, and paired reductions are calculated within each period against its explicit control.

Both informed selectors chose 50 μ s slack and all CPUs, identical to control; their paired improvement is exactly zero in this replay. Control P95 is 179.7 ± 102.0 and 155.7 ± 64.4 μ s at 5 and 8 ms (95% Student-*t* half-width across ten blocks). The training interaction contrast is -18.6 μ s; the pre-specified normalized-contrast bootstrap emitted no edge because its interval included zero. We do not turn a nonsignificant contrast into a dependency rule or a semantic-reasoning gain. Acquisition took 6.54 s for training, 8.29 s for retrieval and 10.15 s for calibration; graph construction and selection

Table 3: Follow-up selector replay: mean of ten test-run P95 values. Misses are observed lateness above 200 μ s. Shared rows across selectors are intentional: selectors choosing the same configuration use the same held-out observations.

Period	Selector	P95 (μ s)	Misses
5	control	179.7	13/400
5	empirical min	179.7	13/400
5	graph min	179.7	13/400
5	seeded random	811.6	248/400
8	control	155.7	13/400
8	empirical min	155.7	13/400
8	graph min	155.7	13/400
8	seeded random	642.2	225/400

Table 4: Matched-budget replay: % of 1,000 seeds choosing control/expected paired P95 change at 5 ms (%); last column: mean training evaluations of 1 ms settings at $B = 40$.

Method	$B = 4$	$B = 8$	$B = 16$	$B = 40$	1 ms evals
Random search	62/-55	74/-38	85/-22	98/-3	19.8
ϵ -greedy (RL)	65/-51	86/-20	95/-8	99/-1	3.8
UCB1 (RL)	65/-51	77/-33	91/-12	99/-1	12.0
GP-EI (BO)	65/-51	83/-25	95/-8	99/-1	2.5

took 0.10 s, included in calibration-phase wall time.

Matched-budget BO and RL. Random search, ϵ -greedy Q-learning ($\epsilon = 0.1$), UCB1 (Auer et al., 2002) and Gaussian-process expected-improvement BO (Snoek et al., 2012) each evaluate $B \in \{4, 8, 16, 40\}$ training runs, starting with one per configuration; an evaluation returns the P95 of a recorded run of the chosen configuration, drawn with replacement. $B = 40$ matches the table selector. Each method recommends its lowest observed or posterior mean, scored by the same replay over 1,000 seeds. Control is best at both test periods, so this comparison can reveal only losses (Table 4). All methods converge on control, reaching 98–99% at $B = 40$; with small budgets, noisy single-CPU runs mislead them. BO spends fewest evaluations on the 1 ms settings, whose training runs miss over 90% of deadlines.

Independent labels and gate misses. For this experiment only, the score is a configuration’s training deadline-miss fraction, fixed before calibration; it is not the REST reasoner’s self-consistency score. A run is labeled unsafe when more than 10% of its 40 observed events exceed the predeclared 200 μ s deadline. Thus the label depends on measured timing, independently of a veto or rollback. Every candidate is executed in an isolated owned process,

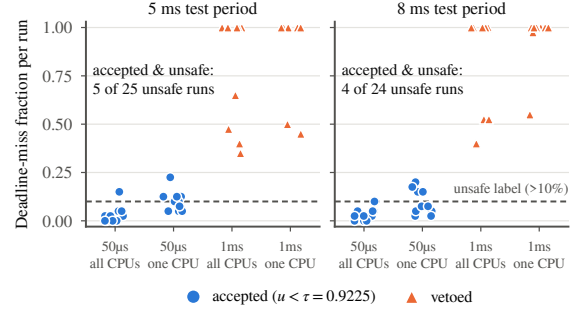


Figure 2: Frozen-gate replay on the 80 held-out test runs; each point is one run. Positive means unsafe (dashed line) and a gate-positive decision is veto. Counts at every tested α (all give $\tau = .9225$): 5 ms, 20 accepted / 20 vetoed, TP 20, FP 0, FN 5, TN 15; 8 ms, 20 / 20, TP 20, FP 0, FN 4, TN 16. Blue points above the line are the accepted unsafe runs. This is a held-out diagnostic, not prevented production harm. Generated from raw logs by `plot_gate_replay.py`.

permitting gate replay with observed outcomes for accepted and vetoed candidates. Calibration uses only 4 ms runs. The three prespecified α values $\{.05, .10, .20\}$ all yield $\tau = .9225$ because the unsafe-score order statistics are tied.

Figure 2 shows every test run. Unsafe acceptance is $5/25 = 20.0\%$ and $4/24 = 16.7\%$ among unsafe runs; unsafe fractions among accepted runs are $5/20 = 25.0\%$ and $4/20 = 20.0\%$. Veto precision is 20/20 in each period, which is not rollback precision. The observed miss rates exceed the nominal 5% and 10% levels. The gate is frozen throughout test, with no retrospective adjustment. The two test periods are different operating conditions, not a drift-recovery trajectory. All counts, thresholds and per-run scores are retained rather than replacing unfavorable outcomes with calibrated claims.

Actual staged application and restoration. A separate adapter snapshots the child thread’s timer slack and affinity, validates the target, applies both settings, verifies readback, and restores both originals in a finally path, including partial-application failures. These per-thread controls are documented Linux interfaces (Linux man-pages project, 2026a,b). Updates are sequential, not an atomic kernel transaction. The driver observes stages of 8, 16 and 32 events and stops when a stage’s deadline-miss fraction exceeds 10%. Stages are observation counts, not fractions of production traffic. All four configurations are exercised in ten blocks at each test period, independently of the gate replay.

Table 5: Model audit: choice with evidence (full, no graph, no retrieval), without it, and after deletion; superscripts count seeds; n/e: non-estimable; [†]hosted. Paired test P95 change versus control at 5 and 8 ms (% , 95% half-width): ctrl 0; 50/1 -145 ± 125 and -180 ± 156 ; 1m/all -599 ± 188 and -631 ± 179 ; 1m/1 -631 ± 183 and -699 ± 206 .

Model	Evidence	None	Del. cited	Del. unc.
Qwen-7B	ctrl	50/1	n/e	n/e
Llama-8B	50/1	50/1	1m/all	ctrl
Qwen-14B	ctrl	50/1	n/e	n/e
Phi-4	ctrl	1m/1	50/1	ctrl
Llama-70B [†]	ctrl	1m/1	ctrl	ctrl
Qwen3-235B [†]	ctrl	1m/1 ³ ctrl	50/1	ctrl
DeepSeek-V3.2 [†]	ctrl	50/1	n/e	n/e
Nemotron-550B [†]	ctrl	ctrl	ctrl	ctrl
Gemini-Flash [†]	ctrl	50/1	n/e	n/e
Gemini-Pro [†]	ctrl	50/1	n/e	n/e

Across 80 fresh executions, 66 stop on a stage breach and 14 complete all stages; all 80 restore and read back both original values. At 5 and 8 ms, observed deadline misses are respectively 186/816 and 190/880; summed exposure durations are 4.106 and 7.066 s. Early stops reduce the number of observed events, so these fractions cannot be compared directly with fixed-duration, no-stop runs to claim an anomaly reduction. The stop criterion itself is not independent ground truth for rollback precision.

Model and explanation audit. The audit supplies only frozen training and retrieval evidence and compares full, no-graph, no-retrieval and no-evidence inputs and cited versus matched uncited deletions. Qwen2.5 7B and 14B (Qwen Team, 2024), Llama-3.1-8B (Grattafiori et al., 2024) and Phi-4 (14B) (Abdin et al., 2024) ran locally as hashed 4-bit GGUF files in llama.cpp with greedy, schema-constrained decoding. Llama-3.3-70B, Qwen3-235B-A22B (Yang et al., 2025), DeepSeek-V3.2 (DeepSeek-AI, 2025), Nemotron-3-Ultra-550B (NVIDIA, 2026), Gemini 3.8 Flash (Google DeepMind, 2026b) and Gemini 3.1 Pro preview (Google DeepMind, 2026a) ran through OpenRouter with the same prompts, schema, seeds and temperature 0; they are unhashable and varied across seeds and providers. 249 of 250 calls were valid.

No model improves on control (Table 5). With evidence, all but Llama-8B choose control like the non-LLM selectors; without it, all but Nemotron (a declared default) and one Qwen3 seed choose a worse setting. Faithfulness is non-estimable for

models that cite the only table. Deletion changes Llama-8B’s choice in both arms and Phi-4’s only for cited items, against the remaining runs; for Llama-70B, Qwen3 and Nemotron it changes neither, consistent with the remaining evidence but uninformative about reliance. Of the quantitative claims we checked in explanations, 16 of 28 (local) and 25 of 69 (hosted, full context) are wrong, mis-attributed, overstated or in the wrong unit. Valid citations establish provenance, not correct reasoning.

6 Conclusion

SemantOS is a research prototype for inspectable, knowledge-grounded kernel-tuning proposals with independent bundle gating. We contribute a bundle-level control contract, correctly conditioned risk statements, and a role-separated evaluation protocol with raw-log verification. Applied to real per-thread kernel interventions, the protocol shows that informed non-LLM selectors and matched-budget BO/RL gain nothing over the default, that a frozen calibrated threshold can miss more unsafe runs than its nominal level under a period shift, that bounded staged actuation restored the original settings in every execution, and that ten LLMs did no better.

Limitations

Scope of the prototype. The REST runtime is a dry run: it never writes kernel settings, and the separate actuator changes only an owned child thread’s timer slack and affinity. The artifact therefore demonstrates a decision contract, a bounded per-thread actuator, raw measurements and non-LLM selector replay, but not a deployed VM-sysctl bundle actuator, traffic isolation or a verified trained checkpoint. We make no deployment-efficacy, production-safety or LLM-superiority claim; Section 5 lists the matched comparisons such claims require. The staged adapter is not wired into the REST VM-sysctl runtime and does not establish traffic isolation or recovery from irreversible effects. The reasoner awaits model samples synchronously, so we make no fast-path latency or resource-footprint claim; a nonblocking deployment would separate synchronous enforcement from asynchronous proposals and version the objective weights, policy, graph and calibration snapshots per decision. Logs are append-only files within a process, without tamper resistance. Prompting a model to obey graph edges does not

enforce those edges; independent validation is required, and the current REST runtime enforces only its explicit example constraints.

Evaluation breadth. All controlled measurements use one periodic workload family on one shared host, with roles separated by period rather than by workload family or hardware. Measured graph induction on the training sweep emits no edge. Selector, BO and RL comparisons replay recorded runs rather than executing closed loops, and control is best on the tested grid, so these comparisons can reveal only losses. Run-level intervals describe variability on a shared host and do not account for uncontrolled interference, warm-cache bias or correlated system state; exploratory intervals are unadjusted for multiple comparisons and temporal dependence, and the closed-loop local load does not characterize queueing under an externally fixed arrival rate. The model audit is retrospective and small, and hosted runs are not reproducible bit for bit. The available telemetry prototype contains scheduler and block-I/O demonstration proxies, including timestamp-derived buckets; those are not end-to-end request latencies and must not calibrate an empirical SLO claim. The local benchmarks record completion, failure and deadline data for their narrow tasks but do not substitute for web serving, TPC-C, Kafka/Spark, GPU inference or audio pipelines.

Risk statements. The rank gate’s assumptions can fail under drift, selective labeling, or a score model updated using calibration data, and the observed miss rates under a period shift already exceed the nominal level. Because calibration and test periods differ, runs share a host, and the proposition is marginal under exchangeability, the gate replay neither verifies its deployment assumptions nor establishes recovery. A canary can suffer harm before detection, and rollback cannot undo every side effect. No no-worse-than-default, catastrophic-regression prevention, Pareto superiority or universal safety claim follows from the prototype. Valid citations in model explanations establish provenance, not faithful or correct reasoning. These limitations define concrete measurements and engineering prerequisites for the next evaluation.

Ethics Statement

The measurements run local software workloads and involve no human participants or personal data.

The REST runtime performs no kernel writes; the experimental adapter changes only its owned child thread’s timer slack and affinity, restores both settings, and performs no host-wide sysctl writes. Broader interventions require an isolated testbed, documented rollback limits and appropriate authorization. Any future human study requires prospective institutional review and informed consent.

AI coding assistants were used in developing parts of the code, prose and analysis. Every reported number is produced by released scripts from raw logs whose SHA-256 hashes are recorded, so that it can be recomputed independently; no generated value substitutes for an unavailable observation.

References

- Marah Abdin, Jyoti Aneja, Harkirat Behl, et al. 2024. Phi-4 technical report. arXiv:2412.08905. <https://arxiv.org/abs/2412.08905>.
- Anastasios N. Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. 2024. Conformal risk control. In *International Conference on Learning Representations*. <https://arxiv.org/abs/2208.02814>.
- Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. 2002. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256.
- Rina Foygel Barber, Emmanuel J. Candès, Aaditya Ramdas, and Ryan J. Tibshirani. 2023. Conformal prediction beyond exchangeability. *The Annals of Statistics*, 51(2):816–845. <https://doi.org/10.1214/23-AOS2276>.
- Albert Bifet and Ricard Gavaldà. 2007. Learning from time-changing data with adaptive windowing. In *Proceedings of the 2007 SIAM International Conference on Data Mining*, pages 443–448. <https://dblp.org/rec/conf/sdm/BifetG07.html>.
- Huilai Chen, Yuanbo Wen, Limin Cheng, Shouxu Kuang, et al. 2024. AutoOS: Make your OS more powerful by exploiting large language models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *PMLR*, pages 7511–7525. <https://proceedings.mlr.press/v235/chen24at.html>.
- Benjamin Cohen-Wang, Harshay Shah, Kristian Georgiev, and Aleksander Mądry. 2024. ContextCite: Attributing model generation to context. In *Advances in Neural Information Processing Systems*, volume 37. <https://arxiv.org/abs/2409.00729>.
- DeepSeek-AI. 2025. DeepSeek-V3.2: Pushing the frontier of open large language models.

863	Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon,	866
864	and Bohan Zhang. 2017. Automatic database man-	867
865	agement system tuning through large-scale machine	868
	learning. In <i>Proceedings of the 2017 ACM Inter-</i>	869
	<i>national Conference on Management of Data (SIG-</i>	
	<i>MOD)</i> , pages 1009–1024. https://doi.org/10.1145/3035918.3064029 .	
	Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V.	870
	Le, Ed H. Chi, Sharan Narang, Aakanksha Chowd-	871
	hery, and Denny Zhou. 2023. Self-consistency im-	872
	proves chain of thought reasoning in language mod-	873
	els. In <i>International Conference on Learning Rep-</i>	874
	<i>resentations</i> . https://arxiv.org/abs/2203.11171 .	875
		876
	An Yang et al. 2025. Qwen3 technical report.	877
	arXiv:2505.09388. https://arxiv.org/abs/2505.09388 .	878
		879
	Ji Zhang, Yu Liu, Ke Zhou, Guoliang Li, et al. 2019. An	880
	end-to-end automatic cloud database tuning system	881
	using deep reinforcement learning. In <i>Proceedings of</i>	882
	<i>the 2019 International Conference on Management</i>	883
	<i>of Data (SIGMOD)</i> , pages 415–432. https://doi.org/10.1145/3299869.3300085 .	884
		885